



UNIDADE II

Programação Orientada
a Objetos Com C#

Prof. MSc. Tarcísio Peres

Conteúdo da Unidade II

Abstração e interfaces

- Classes e membros abstratos; ocultação de detalhes e contratos.
- Interfaces, implementação explícita, substituíbilidade (LSP) com exemplos.

Herança e polimorfismo

- Hierarquias, abstract/sealed, composição como alternativa.
- Despacho virtual: virtual/override, polimorfismo por interface.

Fundamentos

- Elaborar sistemas orientados a objetos de qualidade envolve compreender princípios fundamentais como abstração, encapsulamento e contratos.
- Em C#, a abstração se manifesta por meio de classes abstratas e membros abstratos, enquanto o encapsulamento diz respeito à ocultação de detalhes de implementação.
- Além disso, o conceito de contrato aparece principalmente nas interfaces, que definem conjuntos de funcionalidades que as classes devem prover.
- Esses elementos trabalham em conjunto para produzir códigos mais modularizados, flexíveis e de fácil manutenção, ao separar nitidamente o “o quê” do “como” em uma aplicação.
 - Abstração consiste em focar nos aspectos essenciais de um objeto ou sistema, omitindo detalhes supérfluos.
 - Permite modelar conceitos do mundo real de forma simplificada, apropriada ao problema que se deseja resolver.
 - Ao abstrair um objeto “forma geométrica”, nos interessam propriedades e comportamentos gerais como cálculo de área, mas não detalhes específicos de cada forma concreta.

Fundamentos

- Uma classe abstrata é aquela que não pode ser instanciada diretamente – ou seja, não é possível criar objetos concretos dela.
- Ela existe para servir de modelo ou classe base a outras classes mais especializadas.
- Uma classe abstrata frequentemente contém definições de métodos abstratos, os quais também não possuem implementação e devem obrigatoriamente ser substituídos (override) pelas subclasses concretas.
- Assim, ao declarar um método como abstrato, a classe base está adiando sua implementação para as subclasses, garantindo que cada subclasse forneça sua versão específica do comportamento requerido.
- Esse mecanismo força um contrato entre a superclasse e as subclasses: quem herda de uma classe abstrata se compromete a implementar todos os membros abstratos herdados, assegurando que certos métodos existam na classe derivada.

Fundamentos

- Caso a subclasse não implemente algum método abstrato herdado, ela própria passa a ser considerada abstrata pelo compilador (devendo, portanto, também ser declarada com o modificador abstract).
- Uma classe abstrata pode definir tanto membros abstratos quanto membros concretos (não abstratos). Membros abstratos – que incluem métodos, propriedades, indexadores ou eventos – são declarados com o modificador `abstract` e não possuem corpo; já os membros não abstratos possuem implementação normal.
- É permitido que uma classe abstrata contenha zero métodos abstratos; nesse caso, marcá-la como abstrata serve apenas para impedir sua instanciação direta e possivelmente indicar que ela é incompleta conceitualmente.
- Classes abstratas podem, inclusive, definir campos e construtores.
- Embora não se possa criar instâncias de uma classe abstrata, suas subclasses podem invocar o construtor base (da classe abstrata) para reutilizar lógica de inicialização comum. Assim, classes abstratas possibilitam fatorar código compartilhado e ocultar detalhes de implementação.

Exemplo

- Uma classe abstrata Forma que define um método abstrato CalcularArea() – pois cada forma geométrica concreta tem sua própria fórmula de área – e também fornece alguma funcionalidade comum, como um nome ou uma descrição genérica.

```
// Classe abstrata base
public abstract class Forma
{
    public string Nome { get; protected set; }

    // Construtor da classe abstrata base
    protected Forma(string nome) {
        Nome = nome;
    }

    // Membro abstrato: nenhuma implementação aqui
    public abstract double CalcularArea();

    // Método concreto: implementado na classe abstrata
    public void Descrever()
    {
        Console.WriteLine($"Esta forma é um {Nome}.");
    }
}
```

Exemplo

- Subclasses: Circulo e Retangulo

```
// subclasse concreta da forma
public class circulo : Forma
{
    private double raio;

    public circulo(double raio) : base("círculo") {
        this.raio = raio;
    }

    // Implementação obrigatória do método abstrato
    public override double CalcularArea()
    {
        return Math.PI * raio * raio;
    }
}

// outra subclasse concreta
public class Retangulo : Forma
{
    private double largura, altura;

    public Retangulo(double largura, double altura) :
base("retângulo") {
        this.largura = largura;
        this.altura = altura;
    }

    public override double CalcularArea()
    {
        return largura * altura;
    }
}
```

Exemplo

- Note que não podemos instanciar Forma diretamente. Qualquer tentativa de executar `new Forma("alguma")` resultaria em erro de compilação, pois Forma é abstrata.
- Em vez disso, instanciamos as subclasses concretas.
- As variáveis `f1` e `f2` são do tipo abstrato Forma, mas referenciam instâncias concretas de `Circulo` e `Retangulo`.
- Ao invocar `CalcularArea()` em `f1` e `f2`, o método apropriado de cada subclasse é executado (graças ao `override` polimórfico).
- Benefício-chave das classes abstratas: elas permitem o programar orientado à abstração, manipulando objetos diferentes através de uma interface comum.

```
Forma f1 = new Circulo(2.5);  
Forma f2 = new Retangulo(3.0, 4.0);
```

```
Console.WriteLine( f1.CalcularArea() );    // Chama  
Circulo.CalcularArea()  
Console.WriteLine( f2.CalcularArea() );    // Chama  
Retangulo.CalcularArea()  
f1.Descrever();    // Usa Forma.Descrever() para um círculo  
f2.Descrever();    // Usa Forma.Descrever() para um retângulo
```

Exemplo

- O código que usa Forma não precisa conhecer os detalhes internos de cada subclasse para chamar `CalcularArea()` – ele sabe apenas que toda Forma tem esse método, conforme definido no contrato da classe base.
- Os detalhes do cálculo ($\pi \times r^2$ para círculo, base x altura para retângulo etc.) estão ocultos dentro das respectivas classes concretas.
- Essa ocultação de detalhes de implementação reduz a complexidade e aumenta a flexibilidade do software: podemos adicionar novas formas (ou modificar as existentes) sem alterar o código que trabalha com o tipo abstrato Forma, desde que o contrato (a interface pública) seja mantido.
- A classe expõe apenas o que for necessário através de membros públicos (sua interface pública) e restringe o acesso às partes internas, marcando-as como privadas ou protegidas.
- Combinando encapsulamento com abstração, obtemos um design em que cada classe é uma caixa-preta bem definida – sabemos o que ela faz, mas não necessariamente como o faz internamente.

Interfaces

- Enquanto as classes abstratas fornecem uma forma de abstração dentro de uma hierarquia de herança, as interfaces oferecem outra maneira de definir contratos em C#.
- Uma interface é essencialmente um contrato que garante como uma classe ou uma estrutura irá se comportar.
- Em C#, uma interface é um tipo completamente abstrato, declarado com a palavra-chave `interface`, que contém definições de métodos, propriedades, eventos ou indexadores, mas sem qualquer implementação concreta.
- Ao dizer que uma classe implementa uma interface, estamos afirmando que essa classe concorda em fornecer implementações para todos os membros definidos na interface, ou seja, em cumprir fielmente o contrato especificado por ela.

```
public interface IForma
{
    double CalcularArea();
}
```

Interfaces

- A interface IForma declara apenas a assinatura do método CalcularArea, sem implementação.
- Uma classe que implemente IForma deverá fornecer o corpo de CalcularArea.
- Por exemplo, Triangulo é uma classe concreta que implementa IForma.

```
public class Triangulo : IForma
{
    public double Base { get; set; }
    public double Altura { get; set; }

    public Triangulo(double b, double h) {
        Base = b;
        Altura = h;
    }

    public double CalcularArea() {
        return (Base * Altura) / 2.0;
    }
}
```

```
IForma fig = new Triangulo(3, 4);
Console.WriteLine( fig.CalcularArea() ); // Saída: 6
```

Diferença – classe abstrata e interface

- Classes abstratas fazem parte de uma hierarquia de herança: normalmente representam uma generalização de um grupo de classes relacionadas.
 - Elas podem fornecer implementação comum (código reutilizável) e também definir membros abstratos a serem implementados nas derivadas.
 - Entretanto, cada classe concreta só pode herdar de uma classe base (limitação da herança simples em C#).
- Já as interfaces não implicam relação hierárquica é-um, mas sim definem capacidades que podem ser agregadas a qualquer classe, independentemente de onde ela esteja na hierarquia.
- Por exemplo, podemos ter uma interface `Imprimivel` que define um contrato para objetos que possam ser impressos em papel. Não existe um parentesco entre `Relatorio` (Documento) e `Fotografia` (Imagem) – elas compartilham apenas a capacidade comum definida pela interface.
- Polimorfismo horizontal: classes distintas compartilham comportamentos através de contratos comuns.

Interatividade

Qual alternativa descreve corretamente a diferença central entre uma classe abstrata e uma interface no design em C#?

- a) Uma interface impõe herança simples, enquanto uma classe abstrata permite herança múltipla de classes.
- b) Uma classe abstrata define capacidades sem relação hierárquica, enquanto uma interface sempre estabelece uma relação do tipo é-um.
- c) Uma classe abstrata tende a representar uma generalização em uma hierarquia de herança e pode oferecer implementação comum, enquanto uma interface define capacidades que podem ser implementadas por classes diferentes, mesmo fora da mesma hierarquia.
- d) Uma classe abstrata não pode conter membros abstratos, e uma interface só pode conter código reutilizável.
- e) Uma interface só pode ser implementada por classes que não herdam de nenhuma classe base.

Resposta

Qual alternativa descreve corretamente a diferença central entre uma classe abstrata e uma interface no design em C#?

- a) Uma interface impõe herança simples, enquanto uma classe abstrata permite herança múltipla de classes.
- b) Uma classe abstrata define capacidades sem relação hierárquica, enquanto uma interface sempre estabelece uma relação do tipo é-um.
- c) Uma classe abstrata tende a representar uma generalização em uma hierarquia de herança e pode oferecer implementação comum, enquanto uma interface define capacidades que podem ser implementadas por classes diferentes, mesmo fora da mesma hierarquia.
- d) Uma classe abstrata não pode conter membros abstratos, e uma interface só pode conter código reutilizável.
- e) Uma interface só pode ser implementada por classes que não herdaram de nenhuma classe base.

Interfaces – detalhamento

- Uma interface define um contrato que classes ou structs devem satisfazer por meio da implementação de determinados membros (métodos, propriedades, indexadores ou eventos).
- Diferentemente de classes abstratas, interfaces não possuem estado e tradicionalmente não continham implementação – elas especificam apenas o que deve ser feito, mas não como.
- Uma classe que assina esse contrato (diz-se que implementa a interface) compromete-se a fornecer código para todos os membros definidos pela interface.
- Por exemplo, imagine uma interface IFormaGeometrica com um método CalcularArea().
- Qualquer classe que implemente IFormaGeometrica deverá fornecer sua própria versão de CalcularArea().
- Permite que instâncias de classes distintas sejam tratadas de maneira uniforme: um algoritmo que opera sobre IFormaGeometrica pode receber tanto um objeto Circulo quanto um Retangulo, desde que ambos implementem a interface – garantindo, assim, a substituibilidade polimórfica esperada em orientação a objetos.

Interfaces – detalhamento

- A interface define um conjunto de funcionalidades esperadas e as classes concretas cumprem esse contrato, promovendo um baixo acoplamento e um design mais flexível do software.
- A partir do C# 8.0, interfaces passaram a suportar implementações padrão de métodos, o que permite evoluir uma interface sem quebrar código existente.
- A interface IFormaGeometrica define um contrato para formas geométricas que podem ter sua área calculada.

```
interface IFormaGeometrica {  
    double CalcularArea();  
}
```

Interfaces – detalhamento

- As classes Retangulo e Circulo implementam essa interface, cada uma fornecendo a lógica específica para calcular sua área.

```
class Retangulo : IFormaGeometrica {  
    public double Largura { get; set; }  
    public double Altura { get; set; }  
  
    public double CalcularArea() {  
        return Largura * Altura;  
    }  
}
```

```
class Circulo : IFormaGeometrica {  
    public double Raio { get; set; }  
  
    public double CalcularArea() {  
        return Math.PI * Raio * Raio;  
    }  
}
```

Interfaces – LSP

- O método a seguir calcula a área total de diversas formas geométricas sem nos preocuparmos com as classes concretas de cada uma. O array formas armazena instâncias de Retangulo e Circulo através do tipo da interface IFormaGeometrica.
- O método AreaTotal itera por cada elemento e invoca CalcularArea() sem precisar saber se a forma é um retângulo ou um círculo; isso demonstra a substituíbilidade polimórfica em ação – qualquer objeto que implemente IFormaGeometrica pode ser usado no lugar de outro, respeitando-se o contrato da interface.
- Esse é exatamente o comportamento descrito pelo princípio da substituição de Liskov (LSP).

Interfaces – LSP

```
double AreaTotal(IFormaGeometrica[] formas) {  
    double soma = 0;  
    foreach (IFormaGeometrica forma in formas) {  
        soma += forma.CalcularArea();  
    }  
    return soma;  
}  
  
// Uso do método:  
var formas = new IFormaGeometrica[] {  
    new Retangulo { Largura = 5, Altura = 4 },    // área = 20  
    new Circulo { Raio = 3 }                    // área ≈ 28.27  
};  
Console.WriteLine(AreaTotal(formas)); // saída esperada ≈ 48.27
```

Interfaces – implementação explícita

- Em C#, o modo padrão de uma classe implementar os membros de uma interface é incluí-los em sua definição com modificador de acesso público (implementação implícita).
- Entretanto, a linguagem oferece a possibilidade de implementação explícita de interface, em que a classe fornece a implementação de um membro de interface sem expô-lo em seu contrato público convencional.
- Em uma implementação explícita, o nome do membro da interface é qualificado com o nome da interface. Dessa forma, o membro só pode ser acessado quando a instância do objeto é referenciada através do tipo da interface, e não através do tipo concreto da classe.
- As implementações explícitas são úteis em alguns cenários específicos. Um caso comum é quando uma classe implementa múltiplas interfaces que possuem métodos homônimos ou semanticamente distintos.

Considere o exemplo a seguir, no qual uma mesma classe implementa duas interfaces diferentes com membros de mesmo nome:

Interfaces – implementação explícita

- No exemplo, CumprimentoMulti fornece duas versões do método Saudar(): uma pública (implícita) retornando "Hello" e outra explícita (associada à interface francesa) retornando "Bonjour".

```
interface IGreetingsEn {
    string Saudar();
}
interface IGreetingsFr {
    string Saudar();
}

class CumprimentoMulti : IGreetingsEn, IGreetingsFr {
    // Implementação implícita para inglês:
    public string Saudar() {
        return "Hello"; // Saudação em inglês por padrão
    }
    // Implementação explícita para francês:
    string IGreetingsFr.Saudar() {
        return "Bonjour"; // Saudação em francês somente via
interface IGreetingsFr
    }
}
```

Interfaces – implementação explícita

- Quando chamamos `obj.Saudar()` diretamente, estamos utilizando a versão pública do método (implementação em inglês). Já ao converter o objeto para `IGreetingsFr` e invocar `Saudar()`, executa-se a implementação explícita definida para o francês. Note que, sem o cast para `IGreetingsFr`, não é possível chamar a versão francesa, pois ela não faz parte da interface pública normal do objeto `CumprimentoMulti`.
- Em tempo de compilação, `obj.Saudar()` se vincula à implementação pública (inglês), e apenas referenciando o objeto pelo tipo `IGreetingsFr` acessamos a outra versão.
- Esse comportamento ilustra a essência da implementação explícita: segregar diferentes comportamentos para diferentes interfaces que uma classe implementa, evitando conflitos de nomes e controlando a visibilidade de cada membro.

```
var obj = new CumprimentoMulti();  
Console.WriteLine(obj.Saudar());           // Saída: Hello  
Console.WriteLine(((IGreetingsEn)obj).Saudar()); // Saída:  
Hello (IGreetingsEn usa implícita)  
Console.WriteLine(((IGreetingsFr)obj).Saudar()); // Saída:  
Bonjour (IGreetingsFr usa explícita)
```

Princípio da Substituição de Liskov

- O LSP é o L do acrônimo SOLID, um conjunto clássico de princípios de design de software.
- Formulado originalmente por Barbara Liskov em 1988, ele afirma que, se S é um subtipo de T, então os objetos do tipo T em um programa podem ser substituídos por objetos do tipo S sem que seja necessário alterar o comportamento correto desse programa.
- Em termos mais simples, qualquer instância de uma classe derivada deve poder ser usada no lugar de uma instância da classe base (ou interface) – e o programa deve continuar funcionando adequadamente, preservando as expectativas de comportamento.
- Esse princípio está intimamente ligado à ideia de contrato: a classe derivada (ou classe que implementa uma interface) deve honrar todas as garantias e expectativas estabelecidas pela classe base (ou interface), não introduzindo restrições adicionais nem comportamentos inesperados do ponto de vista do código cliente.
- Uma interface bem projetada ajuda naturalmente a cumprir a substituíbilidade.

Interatividade

Qual afirmação expressa corretamente o Princípio da Substituição de Liskov (LSP)?

- a) Se S é um subtipo de T, então objetos de T não devem ser substituídos por objetos de S, para evitar polimorfismo.
- b) Se S é um subtipo de T, então objetos de T podem ser substituídos por objetos de S sem quebrar o comportamento correto do programa, desde que S preserve o contrato esperado por T.
- c) Se S é um subtipo de T, então S deve impor restrições adicionais para evitar usos indevidos por parte do código cliente.
- d) O LSP exige que classes derivadas nunca sobrescrevam membros da classe base.
- e) O LSP afirma que a herança sempre é a melhor forma de reutilização, independentemente do contrato.

Resposta

Qual afirmação expressa corretamente o Princípio da Substituição de Liskov (LSP)?

- a) Se S é um subtipo de T, então objetos de T não devem ser substituídos por objetos de S, para evitar polimorfismo.
- b) Se S é um subtipo de T, então objetos de T podem ser substituídos por objetos de S sem quebrar o comportamento correto do programa, desde que S preserve o contrato esperado por T.
- c) Se S é um subtipo de T, então S deve impor restrições adicionais para evitar usos indevidos por parte do código cliente.
- d) O LSP exige que classes derivadas nunca sobrescrevam membros da classe base.
- e) O LSP afirma que a herança sempre é a melhor forma de reutilização, independentemente do contrato.

Hierarquias

- Em POO, é comum organizar as classes em hierarquias de herança para modelar relações do tipo é-um entre entidades.
 - Por exemplo, em um jogo poderíamos ter uma classe base genérica Personagem da qual derivam classes específicas como Guerreiro ou Arqueiro.
- Herdar de uma classe base permite que as subclasses reutilizem atributos e métodos comuns, enquanto especializam comportamentos específicos.
 - Essa reutilização por meio de herança é chamada de reúso white-box (caixa-branca), pois a subclasse tem visibilidade sobre a implementação interna da superclasse.
 - A subclasse abre a caixa da classe base e aproveita seu código, podendo alterar comportamentos por meio de métodos sobrescritos (override).
 - A herança facilita o polimorfismo: um método que recebe um Personagem pode aceitar tanto um Guerreiro quanto um Arqueiro, por exemplo, tratando-os de forma genérica.

Hierarquias

- Uma hierarquia muito profunda ou mal planejada pode tornar o código rígido e frágil diante de mudanças (o chamado fragile base class problem).
 - Alterações em classes base podem afetar todas as subclasses, e nem sempre é fácil estender funcionalidade sem modificar código existente.
- O mecanismo de herança simples do C#, como em Java, não permite herança múltipla de classes – ou seja, cada classe pode ter apenas uma superclasse direta.
 - Isso significa que, quando se tenta modelar um conceito que naturalmente teria múltiplos pais, é preciso recorrer a outras técnicas, como interfaces ou composição.
 - Uma classe abstrata é uma classe pensada para ocupar o topo ou o meio de uma hierarquia, definindo um conceito genérico que não pode ser instanciado diretamente.
 - Como já vimos, tais classes frequentemente contêm métodos abstratos, que são declarações de métodos sem implementação, obrigando as subclasses concretas a fornecer uma implementação.

Hierarquias

- Por exemplo, poderíamos declarar `public abstract class Personagem` com um método abstrato `public abstract void Atacar();`.
 - As subclasses `Guerreiro` e `Arqueiro` seriam então obrigadas a implementar esse método `Atacar` de formas específicas (por exemplo, o guerreiro desferindo um golpe de espada, o arqueiro atirando flechas).
 - Classes abstratas permitem fatorar comportamento comum e definir um contrato para as subclasses, garantindo que certas operações existam em todas as especializações.
- Vale notar que uma classe abstrata ainda pode conter implementação concreta de alguns métodos e propriedades, compartilhando código entre as derivadas.
- Ela serve, portanto, como um modelo parcial com lacunas a serem preenchidas pelas subclasses.
- A tentativa de instanciar uma classe abstrata (por exemplo, fazer `new Personagem()`) resulta em erro de compilação – é necessário instanciar uma das subclasses concretas.

Exemplo

- Definimos uma classe abstrata Personagem com um método abstrato Atacar(), e duas subclasses concretas Guerreiro e Arqueiro que implementam o ataque de formas distintas.
- Guerreiro e Arqueiro herdam de Personagem e cada um define sua própria lógica para Atacar().

```
using System;

abstract class Personagem {
    public abstract void Atacar();
}

class Guerreiro : Personagem {
    public override void Atacar() {
        Console.WriteLine("O guerreiro ataca com sua espada!");
    }
}

class Arqueiro : Personagem {
    public override void Atacar() {
        Console.WriteLine("O arqueiro dispara uma flecha!");
    }
}
```

Exemplo

- O polimorfismo permite invocar Atacar() genericamente em um Personagem – em tempo de execução, o método apropriado à subclasse é chamado.

```
class Program {  
    static void Main() {  
        Personagem jogador1 = new Guerreiro();  
        Personagem jogador2 = new Arqueiro();  
        jogador1.Atacar(); // Saída: O guerreiro ataca com sua  
espada!  
        jogador2.Atacar(); // Saída: O arqueiro dispara uma  
flecha!  
    }  
}
```

Sealed

- Em contraste com as classes abstratas (que servem apenas como base), a linguagem C# também permite marcar classes de forma a impedir que outras herdem delas.
- Usa-se a palavra-chave `sealed` para indicar que uma classe não pode ter subclasses. É equivalente à classe final da linguagem Java.
- Por exemplo, se declararmos `sealed class Arqueiro : Personagem { ... }`, o compilador não permitirá derivar nenhuma nova classe de Arqueiro.
- O objetivo de uma classe selada é sinalizar que sua hierarquia termina ali, seja por motivos de design (a classe representa uma entidade específica que não deve ser estendida inadvertidamente) ou por motivos de segurança e desempenho.
- C# permite selar métodos virtuais individuais.
- Uma subclasse pode sobrescrever um método virtual da base e marcá-lo como `sealed override`, indicando que mais nenhuma classe derivada poderá sobrescrevê-lo novamente.
- Essa técnica é útil para congelar personalizações em um determinado nível da hierarquia.

Composição

- A composição de objetos é outra forma fundamental de reutilização e organização de código.
- Em vez de estabelecer uma relação hierárquica *é-um*, a composição estabelece uma relação tem-um (has-a).
- Isso significa que um objeto é construído a partir de outros objetos internos, delegando a eles parte de sua funcionalidade.
- Diferentemente da herança, na composição não há relação de especialização, nem compartilhamento direto de implementação – cada classe composta mantém sua individualidade e expõe apenas o que for necessário via sua interface pública.
- Por essa razão, costuma-se chamar a composição de reúso black-box (caixa-preta), pois o objeto complexo trata seus componentes como caixas-pretas, utilizando-os através de suas interfaces, sem se acoplar aos detalhes internos deles.
- Objetos complexos podem ser construídos agregando objetos menores; essa agregação modela características do mundo real (um carro tem-um motor, tem-um chassi, tem-um sistema elétrico etc.), promovendo um design modular.

Composição

- Uma diretriz clássica de design orientado a objetos é preferir composição em vez de herança sempre que possível.
- Essa recomendação reconhece que a herança, embora poderosa, pode levar a sistemas rígidos e fortemente acoplados, enquanto a composição favorece maior flexibilidade e menor acoplamento.
- A composição torna mais fácil modificar comportamentos em tempo de execução, trocando os objetos componentes, em comparação com a herança, que fixa o comportamento no momento da definição da subclasse.
- Além disso, a composição evita alguns problemas da herança: não sofre das limitações de herança simples (um objeto pode ter múltiplos componentes de tipos diferentes, contornando a falta de herança múltipla) e reduz o risco de efeitos colaterais de mudanças em superclasses.

Exemplo

- Criamos uma classe Arma (abstrata), da qual derivam Espada e Arco, representando diferentes tipos de arma com suas lógicas de ataque.
- A classe Personagem agora não precisa ser abstrata nem ter subclasses para cada tipo – ela tem um campo do tipo Arma e delega a ela a ação de atacar.
- Assim, trocar a arma do personagem muda seu ataque sem ser preciso trocar o objeto Personagem em si.

```
using System;

abstract class Arma {
    public abstract void Atacar();
}

class Espada : Arma {
    public override void Atacar() {
        Console.WriteLine("Golpe de espada!");
    }
}

class Arco : Arma {
    public override void Atacar() {
        Console.WriteLine("Disparo de flecha!");
    }
}
```

Exemplo

- A classe Personagem compõe um objeto do tipo Arma. O herói é criado com uma Espada.
- Ao chamar `heroi.Atacar()`, a chamada é delegada para `Espada.Atacar()`, produzindo a mensagem de ataque com espada.
- Em seguida, mudamos a composição do objeto em tempo de execução: atribuímos uma instância de Arco à propriedade `ArmaAtual` do herói.
- `heroi.Atacar()` resultará na ação definida em `Arco.Atacar()`, ou seja, disparar uma flecha.

Exemplo

```
class Personagem {
    public Arma ArmaAtual { get; set; }
    public Personagem(Arma armaInicial) {
        this.ArmaAtual = armaInicial;
    }
    public void Atacar() {
        if (ArmaAtual != null)
            ArmaAtual.Atacar();
        else
            Console.WriteLine("Personagem desarmado não pode atacar.");
    }
}

class Program {
    static void Main() {
        Personagem heroi = new Personagem(new Espada());
        heroi.Atacar();           // Saída: Golpe de espada!
        heroi.ArmaAtual = new Arco();
        heroi.Atacar();           // Saída: Disparo de flecha!
    }
}
```

Exemplo

- Note que todo esse comportamento foi obtido sem criar subclasses de Personagem – a classe do herói permanece a mesma, e apenas a arma (um componente) mudou.
- O exemplo também mostra que a composição pode trabalhar em conjunto com outros conceitos, como interfaces ou classes abstratas para os componentes – no caso, Arma é abstrata e Espada/Arco são implementações concretas.
- Composição e a herança não se excluem mutuamente – de fato, podem ser complementares.
 - Por exemplo, poderíamos ter uma hierarquia de armas (arma corpo a corpo, arma de longa distância etc.) combinada com diferentes personagens, ou personagens organizados em hierarquia usando componentes intercambiáveis.
 - A herança é indicada quando há uma clara relação conceitual de especialização e quando se deseja aproveitar ou estender a implementação de uma classe existente.
 - Deve-se evitar hierarquias muito profundas ou complexas demais. Problemas clássicos da orientação a objetos, como a explosão combinatória de subclasses, são um sinal de alerta de que a composição fosse uma escolha melhor.

Interatividade

Qual alternativa descreve corretamente quando a herança com uma classe abstrata tende a ser apropriada?

- a) Quando se quer permitir que uma classe tenha duas ou mais superclasses diretas, usando herança múltipla de classes.
- b) Quando se quer modelar uma relação do tipo é-um, reutilizar comportamento comum, e impor que subclasses implementem certas operações por meio de métodos abstratos.
- c) Quando se quer evitar que subclasses vejam detalhes internos da classe base, garantindo reúso do tipo caixa-preta.
- d) Quando se quer instanciar diretamente o tipo mais genérico, evitando criar subclasses concretas.
- e) Quando se quer eliminar o polimorfismo, garantindo que apenas o método da classe base seja executado.

Resposta

Qual alternativa descreve corretamente quando a herança com uma classe abstrata tende a ser apropriada?

- a) Quando se quer permitir que uma classe tenha duas ou mais superclasses diretas, usando herança múltipla de classes.
- b) Quando se quer modelar uma relação do tipo é-um, reutilizar comportamento comum, e impor que subclasses implementem certas operações por meio de métodos abstratos.
- c) Quando se quer evitar que subclasses vejam detalhes internos da classe base, garantindo reuso do tipo caixa-preta.
- d) Quando se quer instanciar diretamente o tipo mais genérico, evitando criar subclasses concretas.
- e) Quando se quer eliminar o polimorfismo, garantindo que apenas o método da classe base seja executado.

Polimorfismo

- Polimorfismo refere-se à capacidade de tratar objetos diferentes de forma uniforme, desde que compartilhem uma característica em comum (por exemplo, um mesmo tipo base ou contrato).
- Ele permite que objetos de classes distintas, mas relacionadas, sejam manipulados de maneira genérica, cada um respondendo de forma específica à mesma ação.
- Em C#, o polimorfismo se manifesta principalmente de duas formas: através de métodos virtuais (mecanismo de despacho virtual em hierarquias de herança) e através de interfaces (polimorfismo por interface).
- Ambas as abordagens habilitam o chamado late binding (vinculação tardia), ou seja, a decisão de qual método executar ocorre somente em tempo de execução, dependendo do tipo real do objeto, ainda que o código tenha sido escrito de forma genérica.
- Dessa maneira, é possível invocar operações em um objeto sem conhecer exatamente sua classe concreta – apenas sabendo que ele cumpre determinada interface polimórfica definida em alguma superclasse ou interface implementada.

Polimorfismo

- Esse recurso aumenta significativamente a flexibilidade e a extensibilidade dos programas orientados a objetos, já que novos tipos podem ser introduzidos no sistema substituindo comportamentos sem a necessidade de modificar o código que invoca esses comportamentos.
- Uma classe base pode declarar que determinado método é virtual, indicando que as classes derivadas podem sobrescrevê-lo para alterar seu comportamento. Um método virtual definido na classe base possui, portanto, uma implementação padrão, possivelmente incompleta ou genérica, que poderá ser sobrescrita (override) por uma classe derivada.
- O C# também permite declarar métodos abstratos em classes base – lembre-se de que um método abstrato é, essencialmente, um contrato que a classe base impõe às subclasses, pois ele não fornece nenhuma implementação na classe abstrata, apenas a assinatura; conseqüentemente, toda classe derivada concreta é obrigada a fornecer uma implementação (override) para esse método.

Polimorfismo

- Em suma, métodos virtuais e abstratos diferem apenas em possuir ou não uma implementação padrão na classe base, mas ambos fazem parte da chamada interface polimórfica da classe.
- É através de membros virtuais (eventualmente abstratos) que a classe base define um conjunto de comportamentos que poderão ser variados nas subclasses.
- Quando uma classe derivada sobrescreve um membro virtual ou abstrato herdado, ela essencialmente redefine como responder a uma mesma chamada feita em nível de classe base. Assim, objetos de subclasses diferentes podem ser tratados como do tipo da superclasse, executando sua versão particular do método sobrescrito – caracterizando o polimorfismo.
- Por padrão, métodos de instância em classes C# não são polimórficos – isto é, a não ser que sejam declarados como virtuais (ou que estejam implementando um método de interface, como veremos adiante), os métodos de uma classe não podem ser sobrescritos em subclasses.

Polimorfismo

- Uma sintática é que métodos virtuais não podem ser declarados como privados nem estáticos, tampouco podem ser marcados como sealed na própria classe base (o selo é o inverso – é aplicado em subclasses para impedir novas sobrescritas).
- Além disso, obviamente, um método com override não pode alterar a assinatura do método virtual correspondente. Essas regras visam manter a consistência do contrato: o método sobrescrito na subclasse deve ter exatamente a mesma assinatura e modificador de acesso do método virtual original (o compilador reforça isso).
- Os métodos virtuais não podem ser marcados como abstract (e vice-versa) – são conceitos mutuamente exclusivos, já que o método abstrato, por definição, não possui corpo algum.
- A seguir, apresentamos um exemplo prático para ilustrar o polimorfismo via métodos virtuais e override.
- Suponha um cenário de um jogo no qual existe uma classe base Character (Personagem) que define comportamentos genéricos de um personagem do jogo.
- Essa classe possui um método virtual chamado Attack() – que representa o ato de atacar um inimigo. A implementação padrão de Attack() na classe base pode ser simples, apenas informando que o personagem ataca de alguma forma genérica.

Exemplo

- As subclasses de Character representam tipos específicos de personagens, por exemplo Warrior (Guerreiro) e Mage (Mago), cada uma sobrescrevendo o método Attack() para realizar o ataque de maneira distinta, conforme sua especialização.

Exemplo

```
class Character {
    public string Name { get; set; }
    public Character(string name) {
        Name = name;
    }
    public virtual void Attack() {
        Console.WriteLine($"{Name} ataca de forma genérica.");
    }
}

class Warrior : Character {
    public Warrior(string name) : base(name) { }
    public override void Attack() {
        Console.WriteLine($"{Name} avança com sua espada, desferindo um golpe poderoso!");
    }
}

class Mage : Character {
    public Mage(string name) : base(name) { }
    public override void Attack() {
        Console.WriteLine($"{Name} lança uma bola de fogo ardente contra o inimigo!");
    }
}
```

Exemplo

- Character declara o método Attack() como virtual, permitindo que subclasses o sobrescrevam.
- Warrior e Mage herdam de Character e cada um implementa sua versão própria de Attack().
- No método Main(), criamos um array de Character contendo instâncias de Warrior e Mage.
- Ao iterar por esse array, a chamada member.Attack() – na qual member é do tipo da classe base Character – será resolvida em tempo de execução para invocar a implementação específica daquele objeto: se o objeto for do tipo Warrior, executará o código sobrescrito em Warrior.Attack; se for do tipo Mage, executará Mage.Attack.

```
class Program {  
    static void Main() {  
        Character[] party = new Character[2];  
        party[0] = new Warrior("Arus");  
        party[1] = new Mage("Luna");  
        foreach (Character member in party) {  
            member.Attack();  
        }  
    }  
}
```

Exemplo

- Embora a decisão de qual implementação executar seja adiada para o tempo de execução, a resolução de quais métodos existem e podem ser chamados (com base no tipo estático da referência) ocorre em tempo de compilação.
- O C# é uma linguagem estaticamente tipada: o compilador verifica os membros disponíveis a partir do tipo declarado da variável.
 - No nosso exemplo, sabemos que member é um Character e possui um método Attack(). O que o compilador não sabe – e nem precisa saber – é qual classe derivada específica member referencia em um dado momento. Essa escolha fica para o runtime decidir.
 - Em outras palavras, o código é compilado esperando invocar algum Attack() válido em Character, mas só em execução é determinado se essa invocação pula para Warrior.Attack ou Mage.Attack (ou para a versão base, caso não houvesse override).
 - Esse comportamento exemplifica a essência do polimorfismo de subtipo em linguagens OO: o objeto sabe se autosselecionar para executar o método apropriado à sua classe real.

Polimorfismo por interface

- Além do polimorfismo baseado em herança de classes, o C# fornece também o polimorfismo por interface.
- Em termos conceituais, uma interface define um contrato ao qual uma classe (ou struct) pode aderir. Quando uma classe implementa uma interface, ela se compromete a cumprir aquele contrato, ou seja, a prover implementações concretas para todos os membros declarados na interface.
- Todos os membros de uma interface são implicitamente públicos e abstratos – por isso, na definição da interface, não se coloca modificador de acesso nem corpo nos métodos/propriedades.
- Qualquer tentativa de incluir implementação dentro de uma interface tradicional resultaria em erro de compilação.
- Historicamente, interfaces em C# serviam estritamente como contratos puros, definindo o que deve ser feito, mas nunca como.

Polimorfismo por interface

- Quando uma classe ou uma estrutura declara que implementa uma interface (usando o símbolo : na definição da classe, seguido do nome da interface), ela deve obrigatoriamente fornecer a implementação de todos os membros definidos nessa interface – caso contrário, a classe deve ser marcada ela própria como abstract (incompleta).
- Em outras palavras, implementar uma interface é uma operação do tipo tudo ou nada: o tipo que se compromete com uma interface não pode escolher alguns membros para implementar e ignorar outros; ele precisa escrever todos, mesmo que alguns simplesmente deleguem a outras lógicas internas.

Polimorfismo por interface

- Suponha que desejamos modelar objetos em um jogo que podem sofrer dano (por exemplo, personagens e elementos do cenário destruíveis).
- Podemos definir uma interface IDamageable (em português, danificável – ou seja, suscetível a dano) que declara um método TakeDamage(int amount), o qual representa a ação de receber um certo montante de dano.
- Duas classes distintas – por exemplo, Player (Jogador) e Barrel (Barril explosivo) – podem implementar essa interface, cada uma à sua maneira.
- O jogador, ao sofrer dano, reduz suas unidades de vida (Health); já o barril, ao sofrer dano, reduz sua durabilidade até eventualmente explodir.
- Essas classes não precisam ter qualquer relação de herança entre si; basta que ambas implementem IDamageable.
- Podemos então escrever código que trate coleções de objetos IDamageable genericamente, aplicando dano a todos, sem se importar se o objeto é jogador ou barril – cada um reagirá de forma distinta.

Exemplo

■ Interface IDamageable

```
interface IDamageable {
    void TakeDamage(int amount);
}

class Player : IDamageable {
    public string Name { get; set; }
    public int Health { get; private set; }
    public Player(string name, int health) {
        Name = name;
        Health = health;
    }
    public void TakeDamage(int amount) {
        Health -= amount;
        Console.WriteLine($"{Name} recebeu {amount} de dano. Saúde restante: {Health}");
        if (Health <= 0) {
            Console.WriteLine($"{Name} foi derrotado!");
        }
    }
}

class Barrel : IDamageable {
    public int Durability { get; private set; }
    public Barrel(int durability) {
        Durability = durability;
    }
    public void TakeDamage(int amount) {
        Durability -= amount;
        Console.WriteLine($"Barril sofreu {amount} de dano. Durabilidade restante: {Durability}");
        if (Durability <= 0) {
            Console.WriteLine("O barril quebrou em pedaços!");
        }
    }
}
```

Exemplo

- Em Main(), criamos um array do tipo IDamageable e armazenamos um Player e um Barrel nessa coleção – note que isso é permitido porque ambos os objetos são IDamageable, ou seja, satisfazem o contrato da interface.
- Ao iterar sobre targets e chamar target.TakeDamage(30), o programa está, de forma polimórfica, invocando o método da interface sem saber qual é a classe concreta de target em cada iteração.
- Graças ao polimorfismo, cada chamada a TakeDamage será direcionada, em tempo de execução, à implementação correspondente à classe real do objeto: na primeira iteração, target é um Player, então se executa Player.TakeDamage; na segunda, target é um Barrel, então se executa Barrel.TakeDamage

```
class Program {  
    static void Main() {  
        IDamageable[] targets = new IDamageable[2];  
        targets[0] = new Player("Hero", 100);  
        targets[1] = new Barrel(50);  
        foreach (IDamageable target in targets) {  
            target.TakeDamage(30);  
        }  
    }  
}
```

Interatividade

Qual alternativa descreve corretamente o polimorfismo em C# e as formas principais pelas quais ele aparece?

- a) Ele ocorre apenas por herança, e a escolha do método executado é sempre feita em tempo de compilação.
- b) Ele é possível apenas com métodos abstratos; métodos virtuais não participam de interface polimórfica.
- c) Ele exige que todas as classes tenham herança múltipla de classes, para que um objeto possa ter vários tipos base ao mesmo tempo.
- d) Ele permite tratar objetos diferentes de forma uniforme quando compartilham um tipo base ou contrato, e aparece principalmente por métodos virtuais ou abstratos em hierarquias e por interfaces, com decisão do método em tempo de execução.
- e) Ele impede que novos tipos sejam introduzidos sem alterar o código cliente, pois exige conhecer a classe concreta do objeto.

Resposta

Qual alternativa descreve corretamente o polimorfismo em C# e as formas principais pelas quais ele aparece?

- a) Ele ocorre apenas por herança, e a escolha do método executado é sempre feita em tempo de compilação.
- b) Ele é possível apenas com métodos abstratos; métodos virtuais não participam de interface polimórfica.
- c) Ele exige que todas as classes tenham herança múltipla de classes, para que um objeto possa ter vários tipos base ao mesmo tempo.
- d) Ele permite tratar objetos diferentes de forma uniforme quando compartilham um tipo base ou contrato, e aparece principalmente por métodos virtuais ou abstratos em hierarquias e por interfaces, com decisão do método em tempo de execução.
- e) Ele impede que novos tipos sejam introduzidos sem alterar o código cliente, pois exige conhecer a classe concreta do objeto.

ATÉ A PRÓXIMA!